

Research Note: The LLD Practice Problem and Product Direction

Low-Level Design practice platform · research summary · Bishal Chandra Debnath · September 2026

1. The learner problem

Low-Level Design is one of the few interview skills where practice is easy to start and notoriously hard to evaluate. A learner can read a problem statement, sketch a Parking Lot, an Elevator, or a Vending Machine in twenty minutes, and still have no reliable signal whether their responsibilities, abstractions, relationships, and trade-offs are actually good. Unlike data-structures problems, there is no test suite to run and no single correct answer to diff against; two strong designs of the same problem can look almost nothing alike. That leaves the learner reviewing their own work with the same blind spots that produced it.

The consequence is a broken practice loop. Learners solve once, read a reference solution, feel the familiar “I understand this now” glow, and move on — without ever receiving feedback on *their* design or seeing whether a second attempt beats the first. The skill being trained is judgement under constraints, but the typical study flow trains recognition of other people’s answers. Nothing tells the learner what a complete answer looks like before they start, or what specifically to change before they try again.

Three pain points follow: feedback is either absent, unauditable, or expensive (self-review, raw ChatGPT answers, or scarce mentors respectively); the format of a “good answer” is implicit, so learners do not know how much structure (classes, relationships, rationale) is expected; and no tool keeps attempt history, so improvement is invisible and motivation decays. A focused product that closes just this loop — attempt, explainable feedback, history, retry — addresses the part of LLD learning that content platforms structurally cannot.

2. Existing approaches

The landscape splits into content-first courses, passive walkthroughs, raw LLM usage, unstructured peer review, and LeetCode-style judges. Each solves part of the problem; none closes the practice loop. The table summarises what a learner actually gets from each, and where it falls short for design practice.

Approach	What it offers	Where feedback works	Key gap
LLD courses (workat.tech, InterviewReady)	Curated problem sets, video solutions, pattern primers	Compare-your-answer with the instructor’s solution	Single reference answer; no per-learner feedback, no history, no submission
Grokking the OOD Interview (Eduative)	Canonical LLD problems with structured text walkthroughs	Self-check against the walkthrough	Passive reading — nothing to submit, nothing evaluated
Raw ChatGPT / Claude sessions	Strong design reasoning on demand, in conversation	Ask-and-receive critique of a pasted design	No rubric, no persistence, non-reproducible; quality depends on prompt luck
Excalidraw / draw.io + peer review	Free-form diagramming plus community critique (Discord, forums)	Occasional high-quality human review	Slow, unstructured, wildly variable; no scoring, no attempt tracking
LeetCode-style online judges	Battle-tested submit-and-evaluate infrastructure	Deterministic test suites for executable code	Designs are not executable — no test suite can exist for an LLD answer

Table 1: How existing approaches serve (and fail) the LLD practice loop

3. Key gaps

Across all five approaches, the same four gaps recur. First, **feedback is not explainable**: either it does not exist (content), cannot be audited (a raw LLM answer with no evidence), or arrives as an instructor's different design with no bridge to the learner's own choices. Second, **there is no structured submission format**: LLD answers have no equivalent of a code submission, so nothing downstream of practice can be persisted, compared, or evaluated. Third, **attempt history is missing everywhere**: without immutable past attempts and per-attempt scores, improvement is invisible and deliberate practice is impossible. Fourth, **no product combines deterministic checks with LLM reasoning** — the reproducible part of evaluation (requirement coverage, structural smells) and the judgement part (design intent, coaching) are never composed; products choose one extreme or neither.

A fifth, softer gap: existing material rarely makes the evaluation criteria visible *before* the attempt — learners discover what matters only by reading a solution. Surfacing the rubric up front converts practice from guessing into a targeted exercise, and costs almost nothing to build.

4. Product direction

The MVP direction is a focused practice loop, deliberately not an LMS or assessment system: choose a problem, design it in a structured text format, submit, receive explainable feedback, review history, and try again. Four commitments define it.

- **Structured-text submissions.** The learner writes classes (name, responsibilities, collaborators), relationships (from, to, type, why), and a design rationale — a format complete enough to evaluate, light enough to keep the effort on thinking rather than tooling. The rubric is visible before the attempt starts.
- **Hybrid evaluation.** A deterministic rubric engine scores requirement coverage and structural quality (god classes, dangling references, trade-off language), attaching evidence quoted from the learner's own submission; an optional LLM layer adds advisory coaching on top and can never change the reproducible scores.
- **Immutable attempt history.** Submitted answers are never editable; "try again" means a new attempt, so scores over time are trustworthy evidence of progress rather than moving targets.
- **Failure as a first-class state.** If evaluation is slow or fails, the attempt records a recoverable FAILED state with the reason and a retry action — the loop never silently loses a learner's work.

The first release targets a single learner, three seeded problems (Parking Lot, Vending Machine, Elevator), and a zero-dependency monolith — enough surface to prove the loop end-to-end. Every additional system (auth, databases, queues, more problems) is a known extension behind an interface, not a prerequisite.

The gap is not content — it is the loop. LLD learners do not need another solution to read; they need their own answer taken seriously: evaluated against a visible rubric, with evidence, kept in history, and re-attemptable.